

SEM 软件工程管理说明

项目名称：基于大模型的建筑能源智能管理与运维优化系统

文档版本：V1.0

编写日期：2026-05-31

1. 文档目的

本文档对应课程期末项目要求中的软件工程管理内容，说明项目范围、项目计划、团队分工、过程监控与控制、质量管理、风险管理、配置管理和最终交付管理。

2. 项目范围管理

2.1 本期范围

本期必须完成的范围：

- 建筑能耗样例数据集与数据字典。
- 后端 REST API：数据查询、统计分析、导出、问答、工单。
- 前端 Web 工作台：总览、数据浏览、统计分析、工单、决策报告、AI 助手。
- 三维楼层风险态势视图。
- 异常解释和异常工单闭环。
- 外部大模型可选接入。
- MCP Server：数据接入、查询统计、异常解释、运营报告和智能问答。
- 自动化测试、启动脚本、自检脚本和最终交付文档。

2.2 非本期范围

以下内容作为后续扩展，不作为期末项目必须完成项：

- 真实传感器和楼宇自控系统接入。
- 多用户登录、权限、审计日志和组织管理。
- 企业级数据库迁移和高可用部署。
- 正式机器学习模型训练平台。
- 长周期真实节能效果审计。

明确范围边界的原因是避免项目无限扩张，保证课程周期内完成可运行、可验收、可演示的系统。

3. 团队分工

成员	主要职责	核心产出
许奕	项目规划、任务拆解、整合、测试、最终文档和演示打磨	README、任务说明、集成总结、最终文档、系统联调
王天一	后端与架构	REST API、分析服务、MCP Server、后端测试
马源胜	数据与 AI	样例数据、知识库、数据源调研、外部模型配置
周由	前端与集成	Vue 页面、图表、三维风险视图、前后端联调

实际执行中采用“先搭骨架、两轮分工、两次整合、最终封版”的方式，减少多人同时改同一文件造成的冲突。

4. 项目计划

4.1 阶段划分

阶段	目标	主要产出
项目初始化	搭建仓库骨架，明确需求和分工	README、docs、backend/frontend/data/knowledge_base 结构
第一次任务	三人分别完成后端、数据 AI、前端基础能力	第一次任务/ 任务文件和初始实现
第一次整合	合并三方成果，修补缺口	docs/11-first-integration-summary.md
第二次任务	面向完整演示补齐功能	第二次任务/ 任务文件
第二次整合	完成主要功能闭环	docs/12-second-integration-summary.md
封版优化	增强 3D、工单、统计分析、LLM、MCP 和文档	最终系统、验收报告、SRS/SDS/SEE/SEM
最终提交	测试、打包、PPT、视频、Canvas 提交	zip 包、PPT、演示视频

4.2 计划控制原则

- 先完成可运行主链路，再补充高级展示。
- 优先保证数据、后端、前端三层接口一致。
- 每次整合后必须能运行系统。

- 最终阶段不再分配大任务，集中做质量收敛和文档补齐。

5. 软件过程监控与控制

5.1 Git 控制

- 使用 GitHub 作为统一仓库。
- 每轮任务完成后由负责人拉取、检查、整合。
- 真实 `.env` 不提交，防止 API Key 泄露。
- 重要节点形成 commit，便于回滚和追踪。

5.2 任务控制

项目通过 第一次任务/ 和 第二次任务/ 明确任务边界：

- 每个人负责相对独立的目录和模块。
- 提前说明需要阅读的文档、接触的文件和交付物。
- 由整合者统一处理跨模块联调，减少重复冲突。

5.3 接口控制

- REST 接口以 `docs/06-api-contract.md` 为契约。
- MCP 接口以 `docs/16-mcp-integration.md` 为说明。
- 前端调用统一封装在 `frontend/src/lib/api.js`。
- 后端服务层复用，避免 REST 和 MCP 逻辑分叉。

5.4 自检控制

项目提供 `scripts/check-project.ps1`，用于封版前统一检查：

- Python 语法检查。
- 后端 pytest。
- 前端 Vite build。

当前脚本已修复为失败即退出，避免测试失败但仍显示成功的误判。

6. 质量管理

6.1 质量目标

- 系统能从空仓库环境按 README 复现。
- 后端测试全部通过。
- 前端构建成功。
- 演示流程无阻断。
- 文档与当前系统功能一致。

- 提交材料不包含真实密钥。

6.2 测试策略

测试层次	内容	证据
单元/接口测试	数据、分析、导出、问答、工单、MCP	backend/tests/
构建测试	前端生产构建	npm run build
集成测试	REST + 前端 + MCP	scripts/check-project.ps1
人工验收	页面演示路径和浏览器操作	docs/15-project-acceptance-report.md

当前基线结果：后端 75 passed ，前端构建通过。

6.3 文档质量控制

最终文档采用两层结构：

- 过程文档： docs/01 至 docs/16 ，记录需求、方案、接口、任务和验收过程。
- 最终交付件： docs/17 至 docs/21 ，对应 SRS、SDS、SEE、SEM 和提交指南。

7. 配置管理

7.1 代码配置项

- backend/ ： 后端应用、MCP Server、测试。
- frontend/ ： 前端页面、组件、API 封装。
- scripts/ ： 启动、测试和数据生成脚本。

7.2 数据配置项

- data/samples/energy_records.csv ： 样例数据。
- data/dictionaries/energy_records_dictionary.csv ： 数据字典。
- knowledge_base/ ： 知识库。

7.3 环境配置项

- .env.example ： 提交到仓库，给出配置模板。
- .env ： 本地真实配置，不提交。
- data/runtime/ ： 运行期工单数据，不提交。

8. 风险管理

风险	概率	影响	应对措施	当前状态
成员提交质量不一致	中	高	两轮任务边界清晰，最终统一整合	已控制
合并冲突	中	中	目录分工、减少多人改同一文件	已控制
数据太少或像写死	中	高	扩展到 2,976 条、覆盖 3 月至 6 月、加入楼层和设备分析	已控制
外部模型 Key 泄露	中	高	<code>.env</code> 不提交， <code>.env.example</code> 只放占位	已控制
外部模型不可用	中	中	本地知识库兜底，LLM 可关闭	已控制
MCP 依赖导致测试失败	中	中	<code>requirements.txt</code> 声明 mcp，测试固定 asyncio 后端	已处理
自检脚本误报成功	低	高	修复脚本失败退出逻辑	已处理
PPT/视频未完成	中	中	系统侧完成，提交指南提醒补齐	待 PPT/视频负责人完成
真实部署环境差异	中	中	README 写清依赖、 <code>.env</code> 放置、启动脚本	已控制

9. 沟通管理

项目沟通采用“文档先行 + 仓库同步”的方式：

- 需求和分工写入任务文件。
- 接口变化写入 API 契约。
- 整合结果写入总结文档。
- 最终提交要求写入检查清单。
- 运行方式写入 README。

这种方式适合课程小组项目，可以减少口头沟通遗漏。

10. 交付管理

最终交付应包含：

- 源代码：`backend/`、`frontend/`、`data/`、`knowledge_base/`、`scripts/`。
- 文档：SRS、SDS、SEE、SEM、用户手册、测试计划、验收报告、MCP 集成说明。
- 演示材料：PPT、演示视频或视频链接。
- 提交说明：最终打包结构和运行方法。

打包时必须排除：

- `.env`
- `data/runtime/`
- `frontend/node_modules/`
- `frontend/dist/`
- `.pytest_cache/`
- 个人无关作业文件

11. 当前项目状态结论

从工程管理角度看，本项目已经完成需求、设计、实现、测试、文档和验收的主要闭环。剩余事项主要是课程提交层面的PPT、演示视频和最终 zip 打包，不再需要进行第三轮开发任务分配。

本项目的工程管理结论为：范围清晰、过程可追踪、质量可验证、风险已收敛，具备最终提交条件。

? PDF ?????????? Markdown ?????